



UNIVERSITÀ
DI TORINO

MULTIRESOLUTION TENSOR DECOMPOSITIONS WITH MODE HIERARCHIES

Alessandro Zappatore

UNIVERSITÀ DEGLI STUDI DI TORINO

10/07/2026

Cosa sono i Tensori e Perché Decomporli?

- I **Tensori (array multidimensionali)** sono essenziali per rappresentare dati complessi a molti ordini (N-mode), ad esempio per analizzare reti sociali, dati di sensori, traffico internet e pattern in information retrieval.
- Un tensore di ordine N si definisce formalmente come:
$$\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}.$$
- Le tecniche di multiway data analysis, come le decomposizioni tensoriali, agiscono di fatto come algoritmi di **clustering multidimensionale**, estraendo correlazioni nascoste e pesi considerando contemporaneamente le varie "facce" del dato.
- **Il Problema:** I principali algoritmi di adattamento dei modelli tensoriali sono intrinsecamente iterativi e **molto costosi in termini di tempo e risorse** computazionali.

I Modelli PARAFAC e Tucker

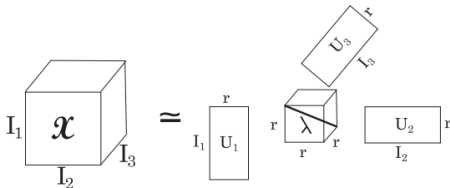
Le due tipologie dominanti di decomposizione, che generalizzano la Singular Value Decomposition (SVD) matriciale per dimensioni elevate, sono:

- **PARAFAC (Parallel Factors/CANDECOMP)**: Decompone il tensore originale in una somma di r tensori di rango 1.

$$\mathcal{X} \approx [[\lambda; \mathbf{U}^{(1)}, \mathbf{U}^{(2)}, \dots, \mathbf{U}^{(N)}]] = \sum_{i=1}^r \lambda_i \mathbf{u}_i^{(1)} \circ \mathbf{u}_i^{(2)} \dots \circ \mathbf{u}_i^{(N)}$$

Viene minimizzata la differenza rispetto al tensore originale:

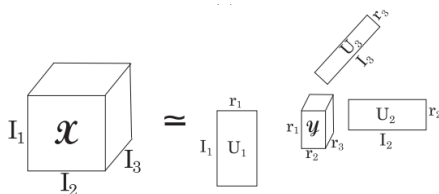
$$\|\mathcal{X} - \sum \lambda_i \dots\|.$$



I Modelli PARAFAC e Tucker

- **Tucker**: Approssima il tensore iniziale usando un "core tensor" di dimensioni più piccole e un set di matrici che rappresentano i sottospazi per ogni dimensione.

$$\mathcal{X} \approx [[\mathcal{Y}; \mathbf{V}^{(1)}, \mathbf{V}^{(2)}, \dots, \mathbf{V}^{(N)}]] = \mathcal{Y} \times_1 \mathbf{V}^{(1)} \times_2 \mathbf{V}^{(2)} \dots \times_N \mathbf{V}^{(N)}$$

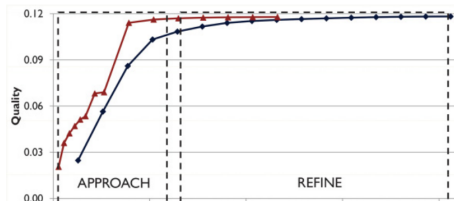


Limiti degli Algoritmi Iterativi (ALS)

- La maggior parte delle librerie si basa sull'algoritmo **Alternating Least Squares (ALS)** per PARAFAC o sull'estensione **HOOI** (Higher-Order Orthogonal Iteration) per Tucker.
- **Come funziona ALS**: Mantiene fisse tutte le matrici dei fattori tranne una; stima e aggiorna quest'ultima matrice per minimizzare l'errore; procede circolarmente alla successiva matrice e reitera il procedimento fino alla convergenza (tolleranza desiderata).
- **Criticità**: Gran parte del tempo di elaborazione è speso nei primissimi passi per calcolare una stima approssimativa a risoluzione completa, operando sempre sull'intero dominio enorme.

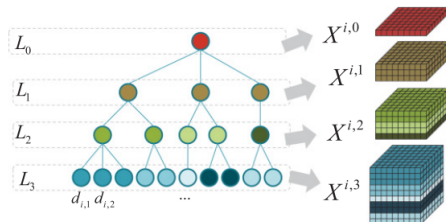
L'Idea alla Base: Multirisoluzione Guidata da Metadati

- Nella maggior parte degli scenari, si dispone di **conoscenza di background** associata alle dimensioni del tensore (ad. esempio: per gli utenti, un clustering basato sulla posizione geografica o l'età).
- **Intuizione:** Usare questi metadati come "guida" per generare versioni del tensore a risoluzione più bassa in base a delle gerarchie predefinite.
- Eseguire la decomposizione inizia su versioni a **bassa risoluzione**, che sono estremamente più veloci da processare.
- Il processo è diviso in **Fase "Approach"** (dove si ci avvicina al target a bassa risoluzione) e **Fase "Refinement"** (in cui si proietta il risultato parziale sul tensore a risoluzione nativa e si applica il fine-tuning finale).



Rappresentazione Tensoriale con Gerarchie

- **Dominio e Gerarchia:** Sia $D_i = \{d_{i,1}, d_{i,2}, \dots, d_{i,I_i}\}$ il dominio dell' i -esima dimensione di un tensore $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$. Si definisce H_i la gerarchia su tale dominio. L'insieme dei nodi a un dato livello l è indicato con $nodes(H_i, l)$. Le foglie coperte dal sottoalbero radicato in uno specifico nodo $n_{i,l,k}$ sono definite dal suo $coverage(n_{i,l,k})$.
- **Representative Selection Function ($repr_i$):** $repr_i : 2^{\mathbb{R}} \rightarrow \mathbb{R}$
Mappa un set di valori relai associati agli elementi del cluster in un singolo scalare rappresentativo (es. min, max, *average*).

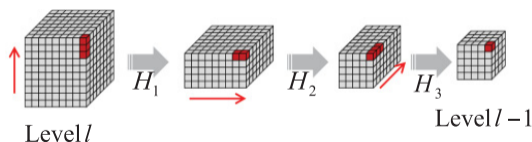


Clustering Embedding Strategy

- **Clustering Embedding Strategy (S):** Definisce formalmente come applicare i metadati al tensore combinando gerarchie, funzioni e livelli scelti:

$$S = \langle \pi, \{(H_1, repr_1, l_1), \dots, (H_N, repr_N, l_N)\} \rangle$$

- π : permutazione degli indici (l'ordine di compressione delle mode).
- H_i (**Mode Hierarchy**): la specifica gerarchia di clustering associata e applicata alla mode i .
- $repr_i$: la funzione rappresentativa scelta per quella dimensione.
- l_i : il livello di gerarchia (risoluzione) adottato per la mode i .



Formule per il Collassamento del Tensore

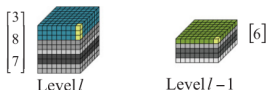
- **Rappresentazione Unimodale:** Riduce la dimensione I_i nel numero di nodi $I'_i = |\text{nodes}(H_i, l)|$. Il tensore compresso a livello l_i è X^{i, l_i} . Se C_h è il cluster corrente della gerarchia, ogni cella viene calcolata come:

$$X_{j_1 \dots j_{i-1} h j_{i+1} \dots j_N}^{(i, l_i)} = \text{repr}_i(\{X_{j_1 \dots j_{i-1} k j_{i+1} \dots j_N} \mid d_k \in C_h\})$$

- **Rappresentazione Multimodale (N-modal):** Estende l'approccio applicando sequenzialmente l'operazione su diverse gerarchie per le varie mode del tensore X , secondo una permutazione definita π .

$$\mathcal{X}_s^S = (\mathcal{X}_{s-1}^S)^{\pi[s], l_{\pi[s]}}$$

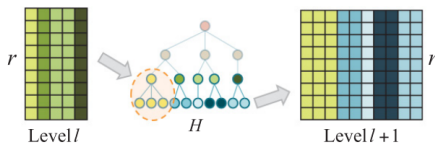
- Più elementi che cadono sotto lo stesso cluster "genitore" vengono fusi usando la funzione media (*average*) limitando drasticamente le iterazioni successive.



Algoritmo 3: Multiresolution ALS per PARAFAC

Il ciclo di vita dell'algoritmo MR si compone delle seguenti fasi:

- 1 Si stabilisce il **livello più basso** di risoluzione richiesto ρ e si comprime il tensore iniziale.
- 2 Si esegue la decomposizione standard ALS sul tensore piccolo, ottenendo un risultato preliminare ($\mathcal{F}(\rho)$) rapidissimo.
- 3 Si avvia un ciclo per risalire i livelli ($resol \rightarrow 0$) fino alla risoluzione massima.
- 4 Le matrici dei fattori precedentemente stimate vengono espanse tramite una funzione di upscaling chiamata `expand_factor()`.
- 5 I fattori espansi diventano il **punto di inizializzazione** della decomposizione al livello superiore.



Algoritmo 4: Multiresolution ALS per Tucker

- Condivide la stessa logica di base impiegata per PARAFAC, ma si innesta sul motore computazionale **HOOI**.
- Vengono processati in input un set di fattori attesi per ogni dimensione $(r_1 \dots r_N)$ invece del singolo rango.
- Calcola inizialmente il core tensor $\mathcal{Y}$ e le factor matrices $\mathbf{V}^i$ sul tensore agglomerato e poi ne esegue l'espansione ad alta risoluzione in maniera iterativa.
- La funzione `expand_factor()` si appoggia su una mappatura "figlio-genitore" decodificata dai metadati: le matrici ritornano alle dimensioni originali per i cicli di sintesi finale.

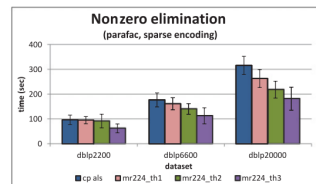
Miglioramento dell'Implementazione: Soglie e Inizializzazione

L'algoritmo gode di specifiche ottimizzazioni flessibili:

- **Inizializzazione delle matrici dei fattori:** Invece di usare valori Random, gli autori propongono un metodo gerarchico o con auto-vettori (v) per mitigare deviazioni drastiche di performance.
- **Allentamento della tolleranza (Relaxed Convergence):** Poiché i livelli intermedi calcolano un'approssimazione grezza, usare **soglie di convergenza larghe** (es. 10^{-2} al posto di 10^{-4}) o eseguire anche solo una singola iterazione di assestamento ai livelli base riduce moltissimo i tempi morti senza alcun impatto negativo sulla decomposizione finale.

La Soluzione per i Dati Sparsi: Zero-Cutoff

- **Problema:** Nei tensori sparsi (ad es. matrici di Social Media molto vuote), condensare i nodi può generare matrici intermedie dense, annullando i benefici di esecuzione.
- **Soluzione:** Rimozione forzata dei non zeri nei livelli intermedi tramite soglia θ_{nz} .
- Viene eliminata dalle stime temporanee ogni cella con valore inferiore a θ_{nz} , riducendo drasticamente il numero di ingressi trattati dalle iterazioni intermedie.
- Questo permette un ulteriore **guadagno del 40%** nel tempo computazionale, con tolleranze qualificative del tutto irrilevanti.



Quality				
	cutoff	dblp2200	dblp6600	dblp20000
cp als		0.055	0.042	0.035
mr224	θ_1	0.053	0.042	0.034
mr224	θ_2	0.053	0.042	0.034
mr224	θ_3	0.053	0.042	0.034

Dataset, Parametri e Metadati Esterni

Per valutare la bontà del framework, l'articolo ricorre a test ripetuti (20 volte per configurazione in ambiente Matlab) e sfrutta:

- **I Dataset:** Dati bibliografici reali da DBLP. Un tensore 3D composto dalle entità Autori $\times$ Conferenze $\times$ Keywords. Versioni estratte sia in formato estremamente Denso che Sparso.
- **Le tre gerarchie:**
 - 1 Conferenze: Estrapolata liberamente da Wikipedia (14 aree madri $\rightarrow$ 27 sottoaree $\rightarrow$ foglia) (Molto esterna).
 - 2 Autori: Partizionamento gerarchico (tramite Metis) su un grafo pre-calcolato di Coautorialità.
 - 3 Keywords: Partizionamento semantico per ricorrenza su articoli simili.

Name	Type	Auth.	Conf.	Keyw.	Entries	Dens.
dblp128	dense	128	500	500	59K	0.186%
dblp256	dense	256	500	500	101K	0.158%
dblp512	dense	512	500	500	166K	0.130%
dblp2200	sparse	2.2K	1K	14K	927K	0.003%
dblp6600	sparse	6.6K	1K	14K	1.9M	0.002%
dblp20000	sparse	20K	1K	14K	3.5M	0.001%

Metriche di Valutazione: Misurare la Qualità della Decomposizione

Valutare in modo oggettivo quanto l'approssimazione calcolata (a qualsiasi livello di risoluzione) sia fedele al tensore originale.

- **PARAFAC:**

$$\text{Qualità} = 1 - \frac{\sqrt{\|\mathcal{X}\|^2 + \|\hat{\mathcal{X}}\|^2 - 2\langle \mathcal{X}, \hat{\mathcal{X}} \rangle}}{\|\mathcal{X}\|}$$

- Dove $\mathcal{X}$ è il tensore originale e $\hat{\mathcal{X}}$ è il tensore approssimato, ottenuto ricomponendo i fattori estratti $\mathcal{F}$.

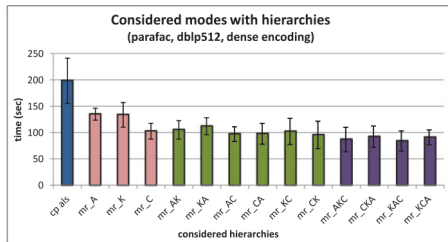
- **Tucker:**

$$\text{Qualità} = 1 - \frac{\sqrt{\|\mathcal{X}\|^2 - |\mathcal{Y}|^2}}{|\mathcal{X}|}$$

- Dove $\mathcal{X}$ è il tensore originale e $\mathcal{Y}$ è il core tensor compresso risultante dalla scomposizione.

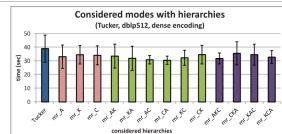
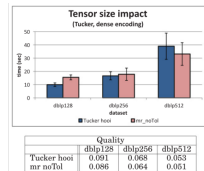
Prove Empiriche e Risparmio per PARAFAC

- L'utilizzo delle gerarchie salva stabilmente fino al 30% del tempo di esecuzione sui tensori densi superando la ALS classica a tutte le "rank target" testate ($r = 5$ a $r = 50$).
- **Multimodalità:** Maggiore è il numero di gerarchie combinate e impiegate simultaneamente nell'encoding, più algoritmo diventa rapido in esecuzione, specialmente unendo le "Autori" e "Keywords".
- **Qualità preservata:** I risultati qualitativi matematici tra modello calcolato con ALS nativo e il framework multirisoluzione si equivalgono, rendendo il boost presentazionale. L'ordine di permutazione (π) delle modalità incise non cambia il risultato.



Risultati Tucker e Supporto ibrido

- Su reti DENSE di dimensioni crescenti, la Tucker multirisoluzione supera notevolmente HOOI standard man mano che la scala cresce, convergendo con ottimi margini e limitando l'errore ai livelli inferiori.
- Inoltre, le gerarchie multiple confermano il trend positivo: usare il mix semantico "Keyword" assieme ad un altro asse porta risparmi del 22% addizionale.
- **Utilizzo Incrociato:** Molto importante è poter calcolare una bse Tucker compressa a Multirisoluzione per **inizializzare la successiva decompressione PARAFAC**, ottenendo il tempo assoluto più basso tra tutti gli scenari sperimentali previsti.



Quality				
Tucker	mr_noTol_A	mr_noTol_K	mr_noTol_C	mr_noTol_AK
0.053	0.051	0.052	0.053	0.051
mr_noTol_KA	mr_noTol_AL	mr_noTol_OL	mr_noTol_OLK	mr_noTol_OLCK
0.051	0.052	0.052	0.052	0.052
mr_noTol_OLCKA	mr_noTol_OLCKA	mr_noTol_OLCKA	mr_noTol_OLCKA	
0.051	0.052	0.051	0.051	